Sygnały i obrazy cyfrowe - Laboratorium
Sprawozdanie
Aiasing 2D, Demozaikowanie, Skalowanie i obrót obrazów rastrowych

Filip Juchnowicz

Indeks 264176

08.01.2023

Spis treści

1 Metody próbkowania sygnałów i obrazów - Aliasing 2D **3**
1.1 Sekwencja obrazów . . . 3
1.2 Sensor . . . 4
1.3 Fragment kodu . . . 5
1.4 Wnioski . . . 5
1.4.1 Możliwe rozwiązania problemu aliasingu . . . 5

2 Zastosowania interpolacji - Demozaikowanie **6**
2.1 Filtry . . . 6
2.2 Interpolacja . . . 7
2.3 Fragment kodu . . . 8

3 Zastosowania interpolacji - skalowanie i obroty obrazów rastrowych **9**
3.1 Obraz oryginalny . . . 9
3.2 Skalowanie . . . 9
3.2.1 Najbliższy sąsiad . . . 9
3.3 Dwuliniowa . . . 10
3.4 Obracanie . . . 10
3.5 Wydajność interpolacji . . . 11
3.6 Fragment kodu . . . 12
3.7 Porównanie jakości . . . 12

Spis rysunków

1 Pojedyńcza klatka śmigła. . . . 3
2 Animacja przedstawiająca pracę skanera dla $l = 8$. . . . 4
3 Zeskanowany obraz dla $l = 4$. . . . 4
4 Fragment kodu odpowiadający za tworzenie klatek kręcącego się śmigła. . . . 5
5 Filtry. . . . 6
6 Zinterpolowany obraz - Bayer. . . . 7
7 Zinterpolowany obraz - X-Trans. . . . 8
8 Kod funkcji interpolacji dwusześciennej. . . . 8
9 Oryginalny obraz o wymiarach 512x512. . . . 9
10 Przeskalowane obrazy - najbliższy sąsiad. . . . 9
11 Przeskalowane obrazy - dwuliniowa. . . . 10
12 Obraz obrócony o 45 stopni. . . . 10
13 Przeskalowane obrazy. . . . 11
14 Czasy wykonywania funkcji. . . . 11
15 Wrapper funkcji odpowiadający za pomiar czasu wykonania. . . . 12

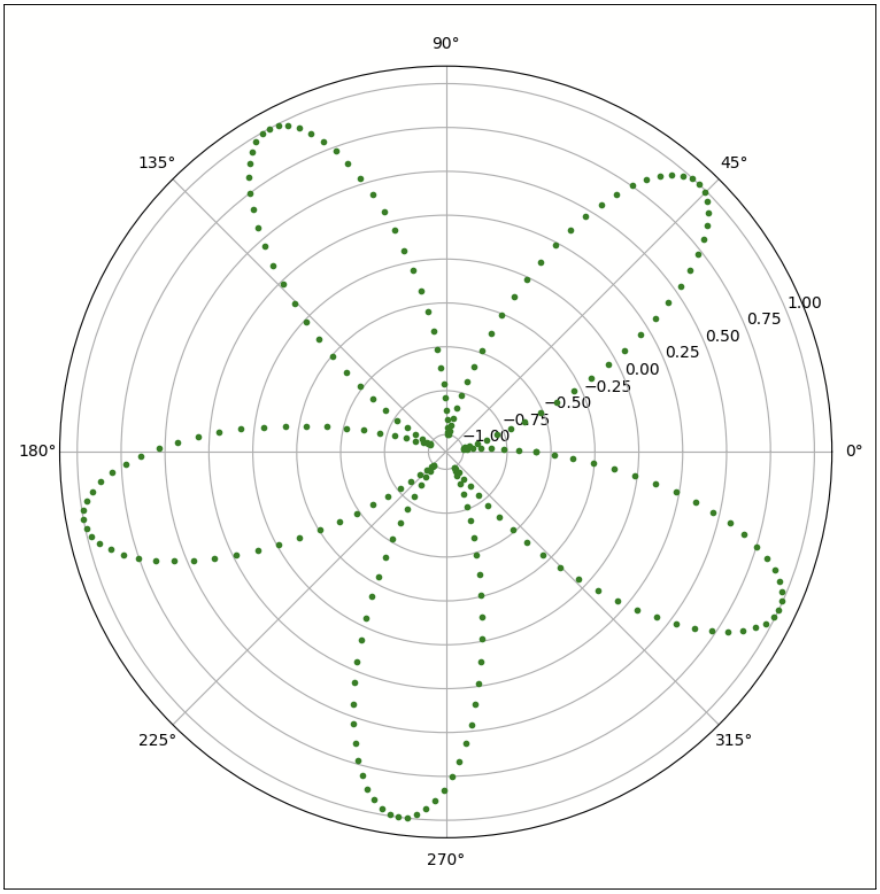
1 Metody próbkowania sygnałów i obrazów - Aliasing 2D

1.1 Sekwencja obrazów

Zadanie polegało na stworzeniu kręcącego się śmigła za pomocą funkcji:

$$f(x) = \sin\left(3x + \frac{m\pi}{10}\right),\ m = -\frac{M}{2},\ \ldots,\ \frac{M}{2}$$

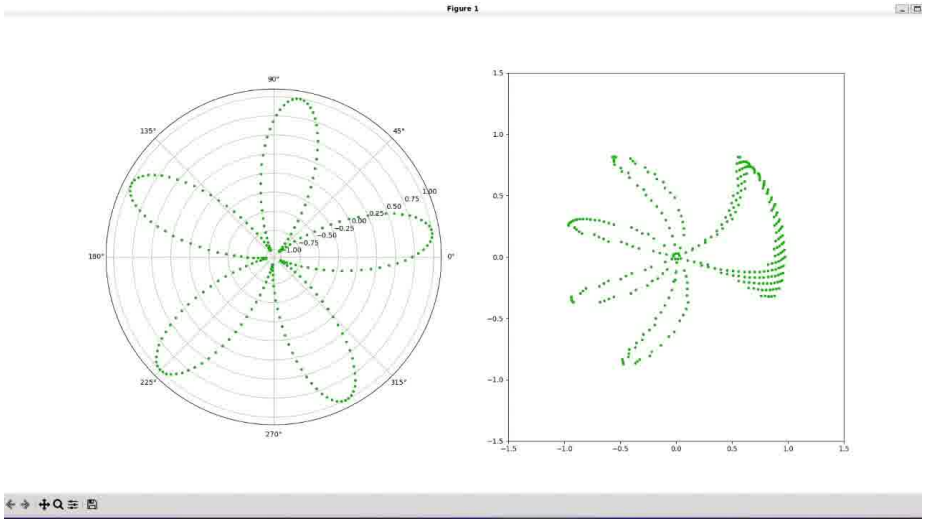
gdzie: n - liczba łopatek śmigła, M - liczba obrazów (klatek).



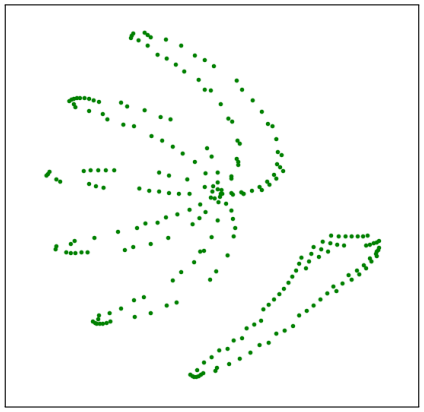
Rysunek 1: Pojedyńcza klatka śmigła.

1.2 Sensor

Następnie należało zasymulować pracę sensora sczytującego l lini (np.: 4, 8, 16). Ze odczytywanych klatek tworzony jest film składający się z $K = 256/l$ (Płynność animacji na sprawozdaniu może nie odwzorowywać rzeczywistego przebiegu filmu).



Rysunek 2: Animacja przedstawiająca pracę skanera dla $l = 8$.



Rysunek 3: Zeskanowany obraz dla $l = 4$.

1.3 Fragment kodu

Ponieważ obraz nanoszony jest na wykres biegunowy zamiast na zmiennych x i y, operujemy na zakresie (radianach) $\langle 0;\ 2\pi \rangle$. Dla zestawu radianów generowane są punkty w oparciu o wcześniej przedstawioną funkcję. Naniesione na wykres biegunowy tworzą pojedyńczą klatkę, a następnie tworzona jest następna klatka, która zastępuje poprzednią tworząc animację kręcocego się śmigła.

```
# Generowanie "obrazu" dla każdej klatki
for frame in frames:
    for rad in radians:
        radius = np.sin(n * rad + (frame * np.pi) / 10)
        # Nanoszenie obrazu na wykres biegunowy
        ax1.plot(rad, radius, "g.")
```

Rysunek 4: Fragment kodu odpowiadający za tworzenie klatek kręcącego się śmigła.

1.4 Wnioski

1.4.1 Możliwe rozwiązania problemu aliasingu

- Chytrym i naiwnym poejściem jest zwiększyć ilość sczytywanych "linii". Im więcej obrazu zapiszemy zanim obiekt się zmieni tym mniej zniekształceń powinno pojawić się na obrazie końcowym.
- Aliasingu można także pozbyć sie podczas postprodukcji, korzystając z:
  - filtru redukcji szumów,
  - narzędzia rozmywania,
  - zmieniając rozmiar obrazu.

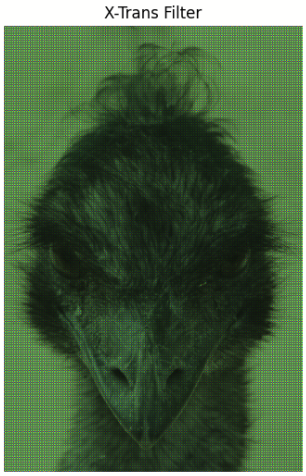
2 Zastosowania interpolacji - Demozaikowanie

2.1 Filtry

Zadanie należało zacząć od demozaikowania, czyli odtworzenie obrazu RGB według schematu Bayera i X-Trans.



(a) Oryginalny obraz wraz z filtrem Bayera.



(b) Filtr X-Trans.

Rysunek 5: Filtry.

Bayer (2x2)

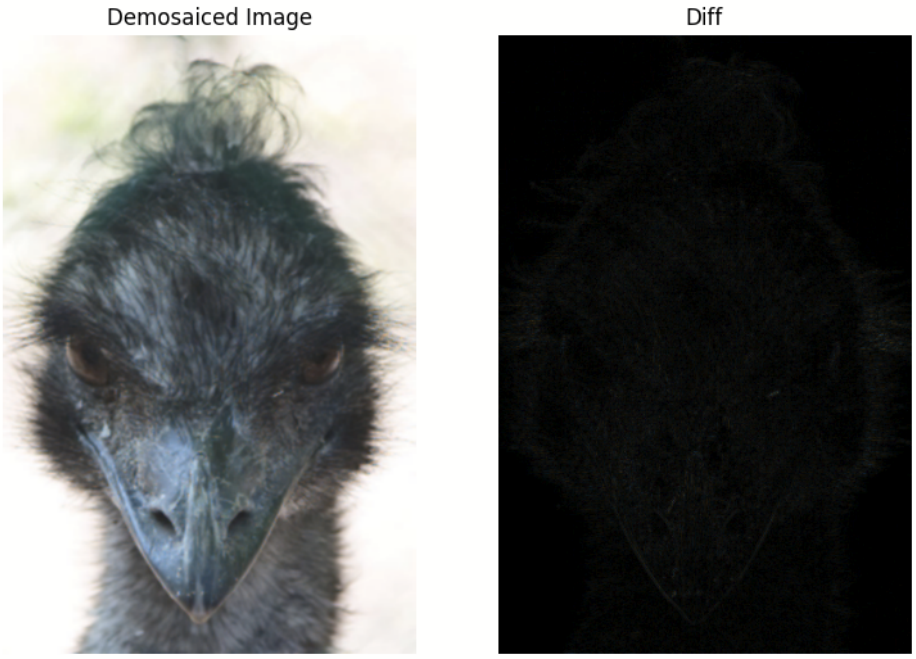
$$\begin{bmatrix} G & R \\ B & G \end{bmatrix}$$

X-Trans (6x6)

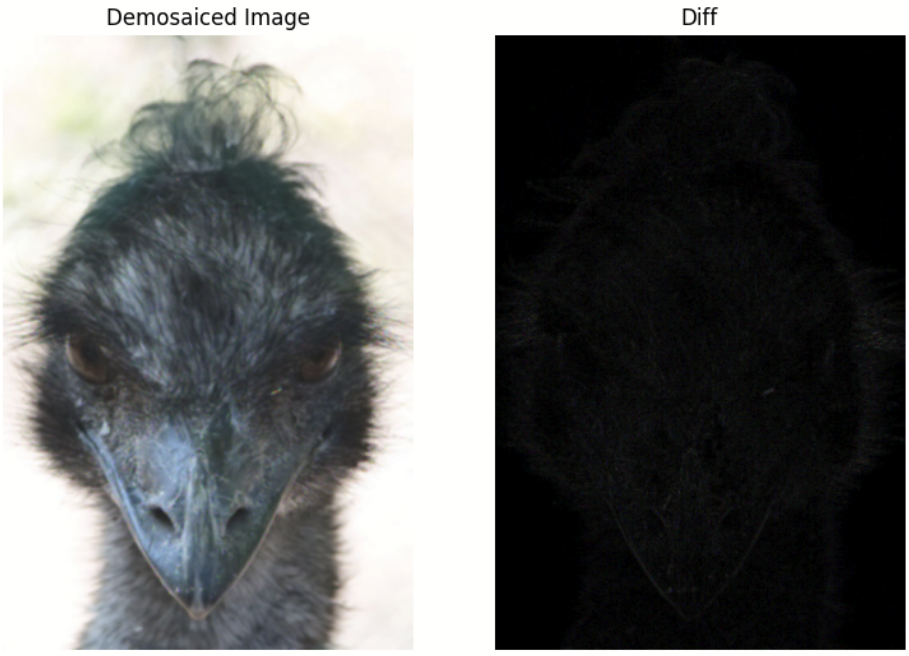
$$\begin{bmatrix} G & B & R & G & R & B \\ R & G & G & B & G & G \\ B & G & G & R & G & G \\ G & R & B & G & B & R \\ B & G & G & R & G & G \\ R & G & G & B & G & G \end{bmatrix}$$

2.2 Interpolacja

Po zdemozaikowaniu obrazu trzeba uzupełnić brakujące piksele wartwach RGB. Zastosowana została interpolacja dwusześcienna (ang. bicubic). Zinterpolowane zdjęcie jest gorszej jakości niż oryginał. Możemy też zauważyć różnice pomiędzy zdjęciami, głównie na krawędziach.



Rysunek 6: Zinterpolowany obraz - Bayer.



Rysunek 7: Zinterpolowany obraz - X-Trans.

2.3 Fragment kodu

Funkcja interpolująca uzupełnia brakujące piksele niezarejestrowane przez matrycę. Liczy ona średnią z ośmiu sąsiadujących pikseli. Patrząc na różnicę w poprzedniej sekcji możemy zauważyć, że obrazy wyglądają podobnie, lecz różnice są bardziej widoczne w zinterpolowanym filtrze X-Trans.

```python
import numpy as np
def demosaic(img, height, width, dim):
    tmp_array = np.zeros([height, width, dim], dtype=int)
    for layer in range(0, 3):
        for y in range(0, height):
            for x in range(0, width):
                cnt, tmp = 0, 0
                for i in range(x - 1, x + 2):
                    for j in range(y - 1, y + 2):
                        if height > j ⩾ 0 and width > i ⩾ 0 and img[j, i, layer] ≠ 0:
                            cnt += 1
                            tmp += img[j, i, layer]
                try:
                    tmp_array[y, x, layer] = tmp // cnt
                except ZeroDivisionError:
                    pass
    return tmp_array
```

Rysunek 8: Kod funkcji interpolacji dwusześciennej.

3 Zastosowania interpolacji - skalowanie i obroty obrazów rastrowych

3.1 Obraz oryginalny



Rysunek 9: Oryginalny obraz o wymiarach 512x512.

3.2 Skalowanie

Następnie należało przeskalować obraz o podaną krotność, potem powiększyć.

3.2.1 Najbliższy sąsiad



(a) Obraz przeskalowany do 256x256 pikseli.



(b) Obraz przeskalowany do 1024x1024 pikseli.

Rysunek 10: Przeskalowane obrazy - najbliższy sąsiad.

3.3 Dwuliniowa



(a) Obraz przeskalowany do 256x256 pikseli.



(b) Obraz przeskalowany do 1024x1024 pikseli.

Rysunek 11: Przeskalowane obrazy - dwuliniowa.

3.4 Obracanie

Obraz został obrócony o kąt 45 stopni. Z powodu problemów z implementacją obliczania nowego wymiaru obracanego zdjęcia, chwilowo część obrazu jest ucinana.



Rysunek 12: Obraz obrócony o 45 stopni.



(a) Obraz obrócony i zinterpolowany funkcją najbliższego sąsiada.



(b) Obraz obrócony i zinterpolowany funkcją dwuliniową sąsiada.



(c) Obraz obrócony i zinterpolowany funkcją dwusześcienną sąsiada.

Rysunek 13: Przeskalowane obrazy.

3.5 Wydajność interpolacji

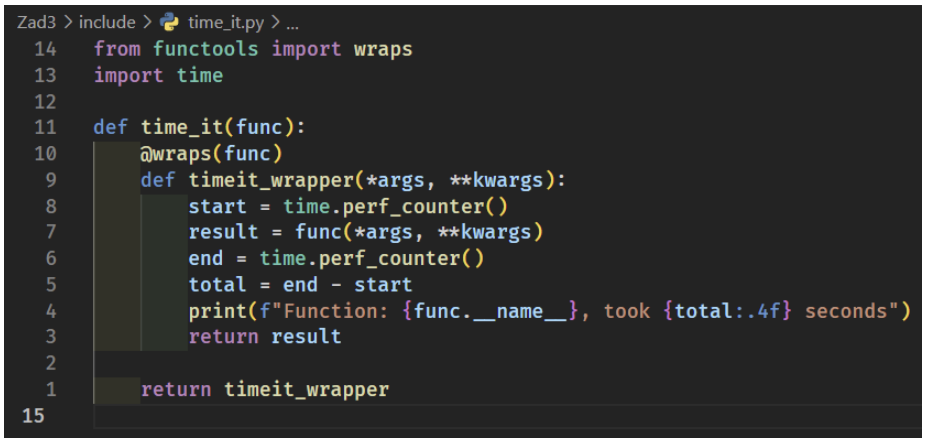
Czasy są kolejno dla:

- Obrotu obrazu o podany kąt,
- Funkcji zmniejszającej o k - najbliższy sąsiad
- Funkcji zwiększającej o k - najbliższy sąsiad
- Funkcji zmniejszającej o k - dwuliniowa
- Funkcji zwiększającej o k - dwuliniowa
- Interpolowanie obróconego obrazu:
  - Najbliższego sąsiada
  - Dwuliniowej
  - Dwusześciennej

```
(venv) morfinix@MorfLapp:~/Documents/PWR-III-sygnaly-cyfrowe/Zad3$ python3 main.py
Function: rotate_image, took 0.1876 seconds
Function: resize_nearest, took 0.3089 seconds
Function: resize_nearest, took 4.9850 seconds
Function: resize_bilinear, took 0.3672 seconds
Function: resize_bilinear, took 4.7756 seconds
Function: nearest, took 4.5140 seconds
Function: bilinear, took 2.1257 seconds
Function: bicupic, took 3.6324 seconds
(venv) morfinix@MorfLapp:~/Documents/PWR-III-sygnaly-cyfrowe/Zad3$
```

Rysunek 14: Czasy wykonywania funkcji.

3.6 Fragment kodu



Rysunek 15: Wrapper funkcji odpowiadający za pomiar czasu wykonania.

Do pomiaru czasu wykonania wykorzystano pythonowskie dekoratory. Są to wrappery, które wykonują jakiś kod przed i po wywołaniu funkcji. Tutaj zaczynają liczyć czas przed i zatrzymują po, a różnica jest czasem wykonania funkcji.

3.7 Porównanie jakości

Zmniejszone zdjęcie wygląda lepiej niż oryginał, lecz przy powiększaniu widać znaczący spadek jakości (krawędzie stają się mniej gładkie). Obrócone zdjęcia są lekko gorszej jakości, gdzie interpolacja najbliższego sąsiada wypada najgorzej.